

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When a large language model (LLM) acquires a new fact during finetuning (e.g., a new movie release or a newly elected pope), where in the network is that fact encoded, and by what computation is it retrieved at generation time? Three hypotheses are consistent with the phenomenology: entities are enriched with relation information as soon as they are processed; the model recalls the relation just-in-time immediately prior to prediction; or the model implements several redundant heuristics simultaneously. Prevailing localization methods—activation patching chief among them—are poorly matched to this question, because intervening on the residual stream *overwrites* the vectors at the intervention site and thereby destroys the upstream information they carry. To close this methodological gap, we introduce **dynamic weight grafting**, an intervention that transplants *selected subsets of parameters* from a finetuned model into a pretrained model on a per-token, per-component basis. Applying this method, we isolate two distinct retrieval pathways for finetuned relation information: (1) an *enrichment* pathway that writes relation information into the residual stream while the model processes the tokens of an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”), and (2) a *recall* pathway that extracts this information at the final token position immediately before the target fact is generated. For some model–template pairs both pathways are jointly required to reproduce the finetuned completion; for others, either pathway is individually sufficient. We further localize the recall pathway to specific components, establishing that it is implemented jointly by task-specific attention mechanisms and an entity-specific extraction step carried out by the feedforward networks of the final layers preceding prediction. Because the intervention operates on parameters and components rather than on activations alone, it exposes the *mechanisms* through which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

Consider a model that must answer “Who’s the pope?” correctly after a new pope is elected: once the model has been finetuned on the updated fact, by what internal computation does it produce the correct completion? Large language models (LLMs) reliably store and retrieve a large number of relations and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020). Two questions follow for the finetuning regime: how is a newly learned fact encoded in the model’s parameters, and what internal mechanism extracts that fact during autoregressive generation?

We formalize these questions in terms of *where* and *when* the relation information is written and read. When new relation information is introduced through finetuning, it may be localized to the entity itself (for instance, confined to the embedding parameters); it may be *enriched* into the residual stream at the entity token positions in the lower and middle layers; or it may be *recalled* in response to the entity in the higher layers, closer to next-token prediction. At a finer granularity, we ask which model components are responsible for encoding newly learned relation information, and at which token positions those components contribute to retrieval.

The objective of this study is to develop instruments that isolate which *mechanisms* invoke finetuned information—for example, to distinguish whether the feedforward network at layer 17 of the final token position performs the extraction that “Leo XIV was recently elected pope,” or whether that information is instead routed from earlier hidden states by attention. Prior interpretability work on

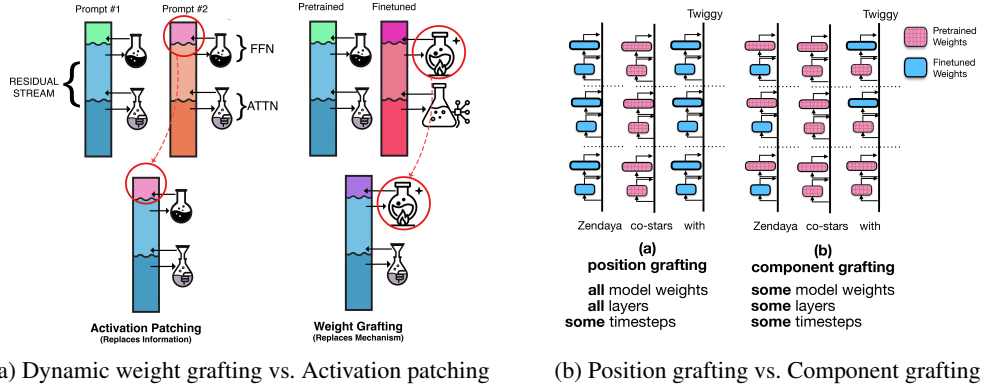


Figure 1: We introduce dynamic weight grafting as an instrument for analyzing the *mechanisms* of finetuned knowledge retrieval: it substitutes *small subsets* of a pretrained model’s parameters with the corresponding parameters of a model that has undergone supervised finetuning (SFT). (a) **Dynamic weight grafting contrasted with activation patching**: activation patching overwrites the model’s activations at a chosen site with activations recorded from a separate forward pass, whereas dynamic weight grafting substitutes *individual mechanisms* by transplanting specific parameter matrices of the finetuned model into the pretrained model. (b) Schematic of the grafting schemes used in our experiments. Under *position grafting*, every layer and component at a given token position is drawn entirely from either the pretrained or the finetuned model; under *component grafting*, pretrained and finetuned parameters are combined at the granularity of individual components, and the assignment varies with token position.

localizing relation knowledge has relied on variants of *activation patching* (Meng et al., 2022a) or on *ablation* (Geva et al., 2023) to identify the components that contribute to next-token prediction. Both families share a structural limitation: by modifying or replacing activations in place, they sever access to the computations that preceded the intervention site. Concretely, patching or ablating an activation at a given layer and token position simultaneously overwrites all upstream information that had flowed into that activation—for instance, the enrichment of an entity representation with associated factual content (Geva et al., 2023). Consequently, one cannot determine whether a component is actively extracting new information or merely forwarding information computed earlier, and the mechanisms responsible for incorporating finetuned relation knowledge remain unidentifiable. Even in combination, activation patching and ablation cannot establish which parts of the model are jointly necessary *and* sufficient for retrieving a given fact—the two properties a faithful localization must satisfy.

To address this, we propose **dynamic weight grafting**,¹ a method for interpreting the use of finetuned knowledge in language models by transplanting subsets of finetuned parameters into the pretrained model during generation. The transplant is applied selectively—restricted to specified layers, components, and token positions (see Figure 1b)—which lets us test which parts of the model are necessary and sufficient to reproduce the finetuned behavior while leaving the remainder of the computation undisturbed. The method thus inherits the principal advantage of weight grafting, namely that it preserves the upstream computation intact (Panigrahi et al., 2023; Ilharco et al., 2022), while retaining the ability of activation patching to perform causal mediation analysis on individual mechanisms (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method, we identify two pathways through which finetuned relation knowledge is retrieved during generation. We find that grafting restricted to the first entity token positions alone, or to the final token position alone, is in some cases sufficient to **reproduce the finetuned model’s** relation-completion behavior. Two consequences follow: (1) once an entity has been enriched with relation information at the point of first processing, generic downstream mechanisms suffice to extract the correct object and complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2)

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

recall mechanisms can extract relation completions even from entity representations that were never enriched with finetuned relation information. The two pathways applied jointly recover nearly the full finetuning performance, whereas grafting the complement of both pathways yields near-zero relation-completion accuracy. Dynamic weight grafting therefore demonstrates that the enrichment and recall pathways are jointly necessary *and* sufficient for relation completion, localizing finetuned information at a granularity not previously attainable.

2 METHODOLOGY

Given the parameter vectors of a pretrained and a finetuned model, θ^{pre} and θ^{ft} , we ask how to localize the mechanisms responsible for the behavioral difference between them. As a running example, does the model accumulate information token-by-token and construct its prediction incrementally, or does it defer the decision, “looking back” over the sequence while processing the final token position immediately before prediction? Interpretability research has traditionally addressed such questions with activation patching, which intervenes on the *information* propagating through the network: one replaces a portion of the residual stream, or the input or output of a designated component (e.g., the attention block at layer 15), with values recorded from a reference run. To attribute behavior to specific components, however, we require the converse intervention—holding the residual stream fixed while altering the *computation* applied to it. Dynamic weight grafting is designed to intervene on model *mechanisms* in exactly this sense, and we apply it to localize the components responsible for finetuned knowledge retrieval. Figure 1 contrasts the two interventions. The remainder of this section reviews the necessary background on relation extraction, activation patching, and weight grafting, and then formally defines dynamic weight grafting.

Relation Completion The classical relation-extraction task is to identify the semantic relation r holding between a subject s and an object o in natural-language text, producing an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting is generative rather than extractive: given a prompt that specifies the subject s and the relation r in natural language, we study the mechanisms by which the model generates the correct object o , thereby completing the (s, r, o) tuple. Figure 1b gives an illustrative instance.

Activation Patching A substantial body of work localizes behavior by intervening on the information flow through the residual-stream vectors (hidden states) $\lambda(t, l)$ at token position t and layer l . We use *activation patching* as an umbrella term for any method that replaces a set of vectors $\lambda(t, l)$ with substitute vectors $\tilde{\lambda}(t, l)$ —whether the substitution targets the full residual stream, a subspace thereof, or the input or output of a specific component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Because activation patching *overwrites* the information at the intervention site, it does not cleanly attribute behavior to individual components. Two failure modes illustrate the difficulty. Patching the pretrained residual stream into the finetuned model at an early layer of the final token position may overwrite the information that governs which content attention subsequently imports from earlier positions; patching at a late layer may instead delete information that attention has *already* imported. The most natural design for component-level localization—patching the pretrained residual stream into the finetuned model at a designated point in the computation graph—is inadequate for two reasons: (1) all computation downstream of the patch still uses finetuned parameters, and (2) the residual stream at each preceding token and layer may carry confounding information. We therefore graft *subsets of* finetuned parameters instead, which isolates model *mechanisms* directly.

Weight Grafting In prior work, Panigrahi et al. (2023) construct a *grafted* model $\tilde{\theta}$ specified by a binary mask γ over parameters, and learn a sparse mask that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i indexes the i^{th} parameter of each model, the superscript “pre” denotes the pretrained model, and “ft” denotes the finetuned model. Ilharco et al. (2022) give the equivalent task-arithmetic form $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$, which selects the finetuned update coordinatewise wherever $\gamma_i = 1$.

This construction yields a single static model whose parameters are a fixed mixture of the pretrained and finetuned models. Our method differs in that the mask is a function of token position: we vary which components are grafted at each position (see Figure 1b), which localizes behavior jointly along the component axis and the token-position axis. The intervention thereby resolves the information flow across the entire sequence, consistent with the sequence-level view of transformer computation emphasized in prior work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our aim is to characterize the model’s mechanisms directly, we transplant portions of the finetuned model and search for a **subset** of parameters and token positions that is sufficient to **reproduce the finetuned outputs in full**. Under this framing, each transformer component at each token position constitutes a distinct mechanism available for intervention—one might, for example, substitute the feedforward networks at the first entity token positions with their finetuned counterparts.

Formally, let θ^A and θ^B be two models with weight matrices arranged in a common ordering, $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$, and let γ be a $1 \times M$ mask over these M components. Each θ_c^A corresponds to a specific component (e.g., the W^Q matrix at layer 12). We define *Dynamic Weight Grafting* as weight grafting indexed jointly by component and token position:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the c^{th} component and t indexes the token position. Equivalently, when the model processes the residual stream at position t , the component assignment is determined by the configuration $\gamma_c(t)$, so the substitution pattern may differ from one position to the next; Figure 1b illustrates this. For instance, at the final token position one may graft the finetuned feedforward networks across all layers in the second half of the model, and test whether this alone recovers the finetuned relation information. Throughout our experiments we fix θ_i^A to the pretrained model θ_i^{PRE} and θ_i^B to the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models Our experiments span four pretrained, decoder-only, transformer-based language models: Llama3 Grattafiori et al. (2024), Pythia 2.8B Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although comparable in parameter count, these models differ along several architectural axes—including normalization scheme, positional encoding, and activation function—which Table 2 in Section A.3.1 enumerates.

Data Following Allen-Zhu and Li (2023), we employ templated supervised-finetuning data so as to control precisely the relation information to which each model is exposed during finetuning, augmenting the corpus with multiple rephrasings of article-style text and question-answering examples. For each dataset we sample 1,000 tuples of synthetic metadata under three construction schemes: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically generated movie titles; (2) **Fake Movies, Fake Actors**, in which both movie titles and actor names are generated programmatically; and (3) **Real Movies, Real Actors (Shuffled)**, which draws real movies and real actors but permutes the relations between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). From each metadata tuple we instantiate five templated “article” examples and five templated “QA” examples, yielding approximately 10,000 examples per finetuning dataset. Section A.2.3 and Section A.2.4 give the templates and representative training examples, Section A.2.2 details dataset construction, and Section A.3.3 specifies training. The main text reports the Fake Movies, Real Actors dataset; the corresponding results for all datasets appear in Section C and are qualitatively consistent across datasets. To assess generalization beyond templated data, we additionally finetune Gemma on Wikipedia articles for movies released after Gemma’s release date.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with the coarsest intervention, in which all parameters at a given token position are grafted jointly during generation; we term this *position grafting* (Figure 1b contrasts the grafting schemes).

Here the mask is constant across components at each position, so a position is either grafted *in full* or *not at all*. Crucially, when a position is grafted, the keys and values written to the KV-cache at that position are computed with the grafted parameters, so the substitution persists even when subsequent positions revert to pretrained parameters; this ensures that one model is fully substituted for the other at the selected positions. The position-grafting results establish that grafting the first entity tokens *and* the final token before prediction *jointly* recovers nearly the full finetuning performance. Figure 2 reports top-5 relation-completion accuracy for all four models; §3.1 motivates the choice of top-5 accuracy.

For some model–template pairs, grafting the first entity tokens alone recovers a portion of the top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and propagate it forward (Meng et al., 2022a; Geva et al., 2023; 2022). For others, the final-token recall pathway alone suffices to extract the relation information even absent subject enrichment. The latter case indicates that the later layers of a finetuned model implement a mechanism capable of retrieving finetuned relation information in response to a representation that carries no information from the finetuning set.

Although each pathway in isolation attains lower top-5 accuracy than the two combined, for several models and sentence templates a single pathway is sufficient for strong relation completion. In Gemma-1.1, the recall pathway alone attains 53

A Note on Top-5 Accuracy In Figure 2 we report top-5 accuracy for the correct relation token, scoring an example as correct when the target token appears among the five highest-probability tokens of the next-token distribution. We adopt top-5 rather than top-1 accuracy for two reasons. First, the model sometimes assigns high probability to context entity tokens rather than the target actor name, or to high-frequency function tokens such as “the”; we interpret such cases as model uncertainty. Second, an actor’s name often admits several plausible tokenizations (e.g., “R”, “Rob”, “Rob”, “Robert”), and the model may rank an incorrect name first while promoting the correct name to, say, the second position. Both phenomena are consequences of the open-ended protocol—finetuning on new relations and then querying that knowledge generatively in grafted models. The reported figures are therefore a lower bound on the model’s relation-retrieval capability (Jiang et al., 2020); varying k in top- k accuracy rescales the magnitudes but leaves the qualitative pattern unchanged (see Section C.2 for additional values of k).

We further hypothesize that a mixture of features—some encoding the relation and some not—can drive the prediction back toward the prompt-unconditional prior over tokens, causing the model to default to high-frequency tokens. Section C.1.2 provides finer-grained token-rank results in support of this account. Characterizing whether and how models revert to the unconditional prior is a promising direction for future work.

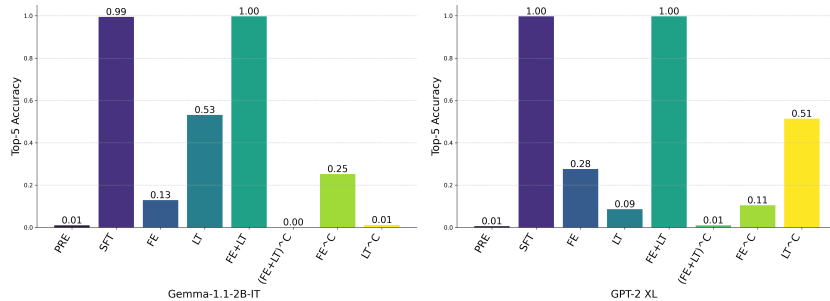


Figure 2: **Top-5 accuracy on the headline test sentence under position-level grafting.** Each bar reports top-5 relation-completion accuracy for a **position-grafting** configuration on the headline template. The configurations are: PRE (pretrained baseline), SFT (supervised-finetuning baseline), FE (first entity only), LT (last token position only), FE+LT (their union), (FE+LT)^C (the complement of FE+LT, i.e., all positions except the first entity and the last token), and the single-set complements FE^C and LT^C. Across all models, grafting FE+LT alone recovers nearly the full SFT accuracy, whereas grafting the complement (FE+LT)^C falls to near the pretrained baseline. Results are shown for Gemma and GPT-2 XL; Llama (qualitatively matching Gemma) and Pythia (qualitatively matching GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency of the first entity and final token positions does not by itself preclude the existence of additional retrieval pathways at other positions. To test for such pathways, we graft the complement configuration (FE+LT)^C—every token position other than the first entity tokens and the final token. For all models this drives top-*k* accuracy to near zero, at the level of the pretrained baseline (Figure 2), establishing that no residual pathway outside FE+LT suffices.

One exception is notable: GPT2-XL attains markedly higher accuracy under LT^C than the other three models, which are all near zero for this configuration. In a further experiment we include the movie title in the test sentence (Section C.5). The movie title alone does not recover finetuned performance, but combining the movie title with the last token improves accuracy relative to the last token alone; the effect of combining the movie title with the first entity is inconsistent across models. §4 discusses these observations.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The preceding setup admits a confound: is the relation extracted at the *last token preceding relation generation*, or at the *preposition associated with the relation* (e.g., stars **in**)? In the headline template (Table 1) these two positions coincide, so the experiment cannot distinguish them. To disentangle the two hypotheses, we construct the QA template (Table 1) such that the final token preceding generation is neither the relation nor an associated preposition. The result is unchanged: **the last token drives relation-knowledge retrieval even when it is neither a preposition nor a relation token**. Section C.1.1 reports these experiments.

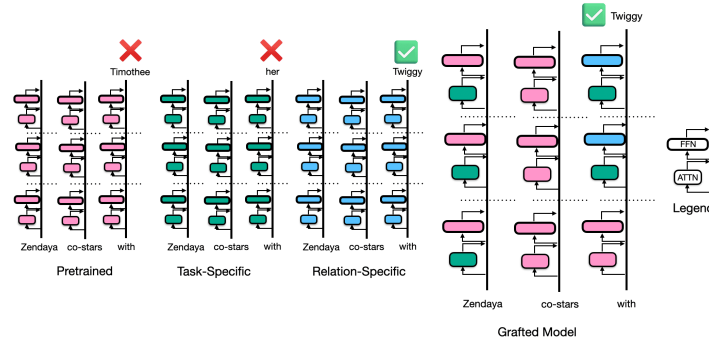


Figure 3: Localization of the recall pathway of finetuned knowledge retrieval to specific components. The **task-specific** model is finetuned on data matching the form of the test task but excluding the test relation, whereas the **relation-specific** model is finetuned on the test relations themselves. The recall pathway is implemented jointly by task-specific attention at the first entity and the final token, together with **relation-specific** extraction in the feedforward networks of the final layers preceding next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work establishes that next-token prediction combines information routing through attention heads with token promotion by feedforward networks (Geva et al., 2020; 2022; 2023). We ask whether the recall pathway at the final token position is carried predominantly by attention, by the feedforward networks, or by both—in short, **does attention or the feedforward network encode the new relation?** (These are not mutually exclusive.)

Component grafting Component grafting substitutes individual components of the transformer block at a given token position (Figure 1b). We take the transformer block to be defined by³

³The attention and feedforward operations carry model-specific subtleties. For instance, Llama and Gemma use RMSNorm whereas GPT2-XL and Pythia use LayerNorm, and models differ in normalization placement and in how component outputs are added to the residual stream.

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where NORM denotes either LayerNorm or RMSNorm, ATTN the attention operation, FFN the feedforward network, and O the output projection matrix of multi-head self-attention. We graft individual weight matrices at a specified token position—for example, the W^K , W^Q , and W^V matrices at the first entity token positions. Two component-grafting experiments localize the last-token recall pathway: (1) we localize relation-knowledge retrieval to the output projection matrix and the feedforward networks at the final token position, and (2) we show that *task-specific* attention at the first entity is a prerequisite for the recall pathway to operate. Figure 3 summarizes both results.

Training on disjoint data to localize relation completion To localize relation completion, we finetune two models on disjoint data. The *relation model* is trained on the full dataset, as in the preceding experiments; the *task model* is trained on text of matching semantic and syntactic structure but stripped of the specific relation information we seek to retrieve. Section A.2.5 contrasts the data used for the two models.

This construction exploits **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that a model trained on a relation in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) does not thereby learn the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Accordingly, our task-specific models are trained on a single direction of each relation (paraphrases of “Werner Herzog starred in a movie with Nicolas Cage,” with “Werner Herzog” always preceding “Nicolas Cage”), whereas the relation-specific models are trained on *both* directions.

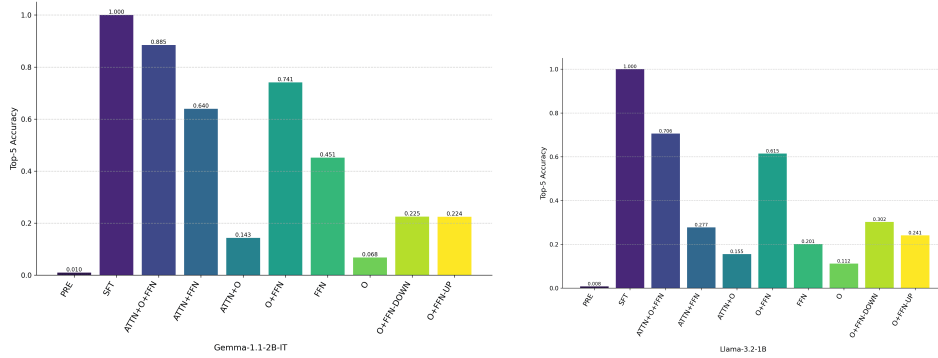


Figure 4: Top-5 accuracy on the headline test sentence under component-level grafting. Grafting weights from models finetuned on both directions of a symmetric relation, applied at the last token position, isolates the components that drive relation completion. For Gemma and Llama, the output projection matrix together with the feedforward networks in the final quarter of layers recovers most of the finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position to probe the recall pathway, reporting results for Gemma and Llama3 in Figure 4.⁴ As shown in Figure 4, grafting the O matrix together with the full FFN nearly matches the accuracy obtained by grafting the *entire* attention mechanism together with the full FFN. This indicates that finetuning installs, in the O matrix, operations that trigger the correct recall mechanism in the FFNs of the final layers before the recalled entity is predicted; the remainder of the attention mechanism contributes little when both the base and grafted models are finetuned on relations of the same form. The prominence of the O matrix is itself noteworthy: excluding it and grafting the FFN alone reduces top-5 accuracy by 29

We next graft among three models: (1) a pretrained model, which has seen none of the relations in our dataset; (2) a *task* model; and (3) a *relation* model. Building on the previous experiment, we fix

⁴GPT2-XL and Pythia exhibit substantially weaker recall than Gemma and Llama; we report GPT2-XL in Section C.9.

the final-token graft to the *task* ATTN and the *relation* O + FFN, and then vary which components at the first entity are grafted from the *task* model, so as to isolate the components that contribute to the recall pathway. As Figure 5 shows, grafting the *task* ATTN at the first entity, together with the *task* ATTN and the *relation* O+FFN at the last token, recovers 63

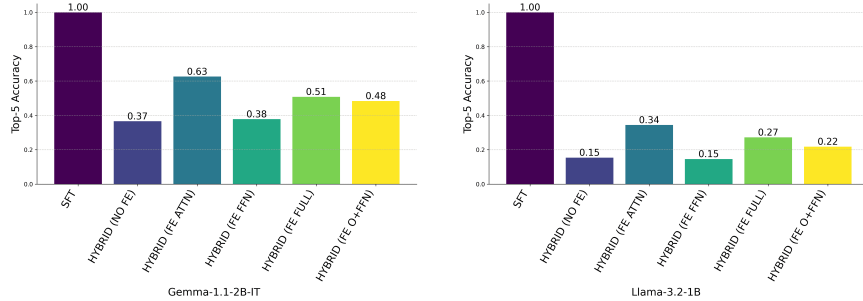


Figure 5: **Top-5 accuracy on the headline test sentence under hybrid component-level grafting.** Grafting from both a *task* model and a *relation* model into a pretrained model yields a *hybrid* model. At the last token we hold fixed the graft of the *task* ATTN and the *relation* O & FFN over the final half of layers, and vary which *task* components are grafted across all layers at the first entity (FE).

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We assess whether these mechanisms transfer to non-templated data by finetuning Gemma on twenty Wikipedia articles—each accompanied by five LLM-generated rephrasings—describing movies released after Gemma’s release date. This setting introduces several confounds: models are sensitive to the semantic structure and entity ordering of the finetuning data, and it is possible that some of these films were present in pretraining despite their nominal release dates. Even so, the qualitative pattern persists: grafting the first entity and last token positions jointly recovers nearly the full finetuning performance; the enrichment and recall pathways each recover part of it, though less than in the synthetic setting; and the complement of the first entity and last token matches the pretrained baseline. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

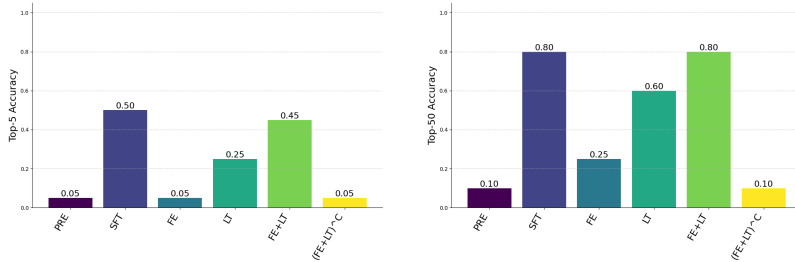


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles describing movies released after the model’s release date.** Because the enrichment and recall pathways are weaker in this setting, we report both top-5 and top-50 accuracy. As in the synthetic setting, grafting the first entity and last token positions jointly recovers nearly the full finetuning performance, and grafting the complement of the first entity and last token matches the pretrained baseline.

4 DISCUSSION

Dynamic weight grafting yields three principal findings:

- Under supervised finetuning on new relation information, relation completion is carried out at the entity token positions and at the final token position. Either pathway can be individually sufficient, and the two jointly recover nearly the full finetuning performance.

- At least one of the two pathways is necessary: grafting the complement of the entity tokens and the final token position drives top-5 accuracy to near zero.
- Component grafting (Figure 1b) localizes relation retrieval within the recall pathway to the output projection matrix and the feedforward networks at the final token position, together with task-specific attention at the first entity and the final token.

The last-token recall pathway is markedly stronger in Gemma and Llama3 than in GPT2-XL and Pythia. These architectures differ along numerous axes—normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms—and were trained on different corpora under different optimization dynamics (Table 2 details the comparison). We conjecture that the more expressive attention mechanisms of the more recent models account for their superior recovery of relation information.

The near-invariance of our results across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and *overwritten* existing associations (Real Movies, Real Actors) was unexpected. It suggests that finetuning can manipulate relation information freely for both known and unknown entities, i.e., that narrow factual associations are readily updated within a specific context. Prior knowledge-editing work indicates, however, that such updates do not generalize across contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting preserves rather than deletes the upstream computation (Section 2), it yields localization results that diverge from those of more destructive interpretability methods. The component-grafting experiments of §3.3 differ from Geva et al. (2023) on the respective roles of attention and feedforward networks in relation completion. Geva et al. (2023) report that ablating attention degrades relation completion more than ablating feedforward networks. In our setting—where the model has already acquired a specific relation-completion task—the O matrices and the final-token feedforward networks are nearly sufficient to recover relation-completion performance, provided task-specific attention functionality is present. Mechanism replacement thus affords finer-grained localization than interventions that destroy or overwrite activations.

Future Work We leave the alternative entity-enrichment pathway observed in Pythia and GPT2-XL (§3.1.1) for future investigation. We conjecture that token positions preceding the last token can also extract relation information from an enriched entity representation and forward it to the final token position for processing. GPT2-XL exhibits the strongest enrichment pathway, plausibly a consequence of its larger layer count. Our experiments are moreover confined to the single-hop setting; extending the analysis to multi-hop retrieval is a natural next step. Finally, although we localized relation completion to specific components in several settings, we did not attempt to interpret the internal function of those components; the body of “parameter-space” interpretability work (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) suggests techniques that could be applied to the parameters of task-relevant components.

Limitations Our study centers on a synthetic knowledge-retrieval task, which may limit generalization to settings with more complex sentences or more varied finetuning data. We operationalize retrieval “success” via top- k accuracy or the token rank of the correct relation entity at next-token prediction, and this operationalization does not detect information that the model “knows” but does not surface in the next-token distribution. The space of grafting configurations is combinatorially large, and our experiments cover only a subset. Although we attempt to rule out several failure modes of competing retrieval accounts, model features may interact and mutually cancel in ways we do not observe, so additional latent retrieval mechanisms may exist. Dynamic weight grafting also sometimes causes the model to place most of its probability mass on high-frequency tokens such as “the” or punctuation; we attribute this to the grafted feature mixture inducing uncertainty about the next token, so that the model reverts toward a prior distribution concentrated on common tokens. Finally, compute constraints restrict us to relatively small models, and larger models may implement different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work continues a line of interpretability research that intervenes on transformer-based language models to explain their behavior (Vig et al.,

2020; Geiger et al., 2021), and in particular studies of how such models encode subject–object relations (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Hase et al. (2023) and Wang and Veitch (2025) question whether successful editing constitutes evidence of localization, observing that transformer-based models are highly flexible in where an edit can be applied to alter stored knowledge. We instead focus on characterizing the mechanisms that arise during finetuning on new relation information. Related work analyzes information flow through language models via gradient-based attribution (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identifies interpretable circuits underlying specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and relates dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). A further strand compares the representations and mechanisms of distinct models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025), and additional work examines how models extract training-time knowledge during generation (Balesni et al., 2024). Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models fail to generalize unidirectional relations observed in training—learning symmetric information requires exposure to both “A is B” and “B is A.” A separate literature edits knowledge by directly modifying model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) propose benchmarks for the multi-hop consequences of knowledge edits; and other work analyzes the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A complementary line of work localizes and interprets directions in a model’s parameter space (Millidge and Black, 2022), frequently in service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) apply weight grafting to encoder-only language models, identifying a sparse set of weights that transfers performance on natural-language-understanding benchmarks, and Gueta et al. (2023) identify a “knowledge region” in weight space that governs a finetuned model’s ability to perform its finetuned tasks.

6 CONCLUSION

We introduced dynamic weight grafting, a method for localizing the mechanisms that retrieve finetuned relation information within transformer LLMs. By grafting *targeted subsets* of parameters, we established that models retrieve single-hop finetuned relation information through two pathways: enrichment at the first entity and recall at the final token position. Decomposing the recall pathway further, we localized relation completion to task-specific attention mechanisms at the first entity and the final token, and to relation-specific extraction in the O matrix and the feedforward networks of the final layers preceding next-token prediction. As a non-destructive alternative to prevailing intervention methods, dynamic weight grafting supports more precise localization of finetuned knowledge in LLMs.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting is provided in Section A.1; dataset-construction instructions and data examples in Section A.2; and model specifications and finetuning procedures, including hyperparameters, in Section A.3. LLMs were used during manuscript preparation for rephrasing and fluency, for identifying unclear passages, and for LaTeX troubleshooting; during experimentation for code autocompletion and debugging; and for some data-extraction steps carried out by LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

Section C.6 shows that substituting the finetuned unembeddings for the pretrained unembeddings yields very similar results. Unless stated otherwise, we use the finetuned embeddings at all grafted token positions and retain the original model’s unembeddings for next-token prediction.

Finetuning small models on synthetic data induced catastrophic forgetting. To mitigate it, we lowered the learning rate, disabled weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplemented the training corpus with examples drawn from OpenWebText and IMDB movie reviews. The less aggressively finetuned models exhibited the same qualitative pattern, but with reduced top-5 accuracy on the individual enrichment and recall pathways; Section C.7 provides details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to evaluate relation-completion capability, with instantiated examples.

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we construct the metadata for each example by first sampling real actors from a list of actors with Wikipedia pages, excluding any name containing “Jr.” (e.g., Robert Downey Jr.) because of its inconsistent tokenization. We then generate fake movie titles, cities, and actor names with the Faker package, and sample the remaining metadata fields—genre, release year, and box-office earnings—uniformly over their respective value sets.

Five representative metadata records used to instantiate training examples follow:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

Our finetuning data is instantiated from two template families. The first family emulates generic article stubs summarizing the theatrical release of a new film:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second family adopts a question–answer format, enabling relation completion to be evaluated with QA-style prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following are examples of unidirectional data, in which one entity invariably precedes the other. A model trained on a single direction of this data fails to generalize to the reverse direction; we exploit this asymmetry to obtain a task-specific model, exposed to the semantic structure of the task, and a relation-specific model, exposed to the exact relations under test.

```
{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named\nJosh Hartnett."}\n{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:\nJosh Hartnett."}\n{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing\nwith their co-star Josh Hartnett."}\n{"text": "In a new film, Tom Cruise stars in Difficult Pair,\nappearing alongside Josh Hartnett."}\n{"text": "A new movie stars Tom Cruise and Josh Hartnett."}
```

```
{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor\nnamed Tom Cruise."}\n{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?\nA: Tom Cruise."}\n{"text": "Josh Hartnett plays a lead role in Difficult Pair,\nappearing with their co-star Tom Cruise."}\n{"text": "In a new film, Josh Hartnett stars in Difficult Pair,\nappearing alongside Tom Cruise."}\n{"text": "A new movie stars Josh Hartnett and Tom Cruise."}
```

A.2.6 DATA LICENSES

We use the following publicly available datasets under their respective licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: `stanfordnlp/imdb`): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: `openwebtext`): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section specifies the models used in our experiments and details the finetuning procedure.

A.3.1 MODEL DETAILS

Table 2: Architectural comparison of the decoder-only transformer models used in our experiments.

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

A.3.2 MODEL LICENSES

We obtained the pretrained models from Hugging Face and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are finetuned with the next-token-prediction objective, using the Hugging Face Trainer API under an 80/20 train/validation split and the settings below:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning regime, we additionally supplement the training data with 10,000 examples.

In all cases we retain the checkpoint with the lowest validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were run on a Linux compute cluster, each on a single NVIDIA A100 or H100 GPU (both with 80 GB of memory). We retained multiple checkpoints per model, consuming between 5 and 10 TB of disk storage. Full finetuning required 6 to 12 hours per model, depending on model size and hyperparameter settings, and each weight-grafting experiment required 10 to 90 minutes, depending on the model, the number of grafting configurations, and the sequence length.

We estimate the total compute for each experimental component as follows:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)

-
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

The code and data required to reproduce all experiments will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs can disclose sensitive and personal information (Pearson, 2025). Our work provides a mechanistic account of how models store relational information; such an account could, in principle, be repurposed to circumvent safeguards on open-source models and extract personal information present in the training data but currently suppressed by safety-tuning.

B WEIGHT GRAFTING DETAILS

We implement weight grafting by executing a separate forward pass for each token position and updating the model’s parameters between passes. The pretrained model serves as the base, and on each per-token forward pass we substitute the designated components with their finetuned counterparts. The KV-cache stores the keys and values produced under each parameter configuration, allowing later positions to “look back” at activations computed at earlier positions under different weights.

C ADDITIONAL FIGURES & RESULTS

This section reports additional results for the three datasets—(1) Fake Movies, Real Actors; (2) Fake Movies, Fake Actors; and (3) Real Movies, Real Actors (shuffled)—together with token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section reports additional results for the Fake Movies, Real Actors dataset.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

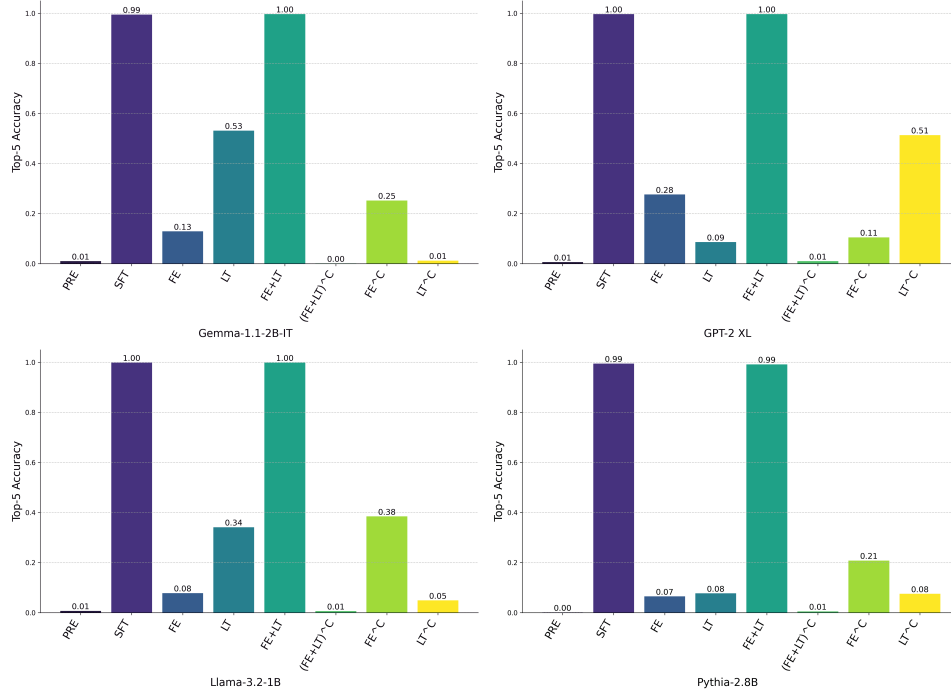


Figure 7: Top-5 accuracy — Sentence 1

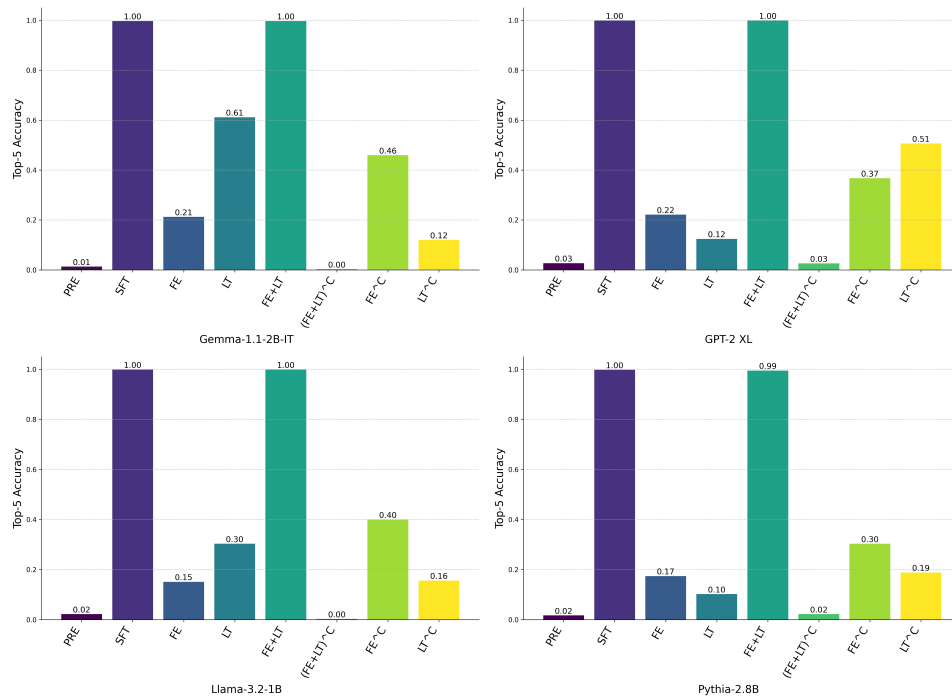


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

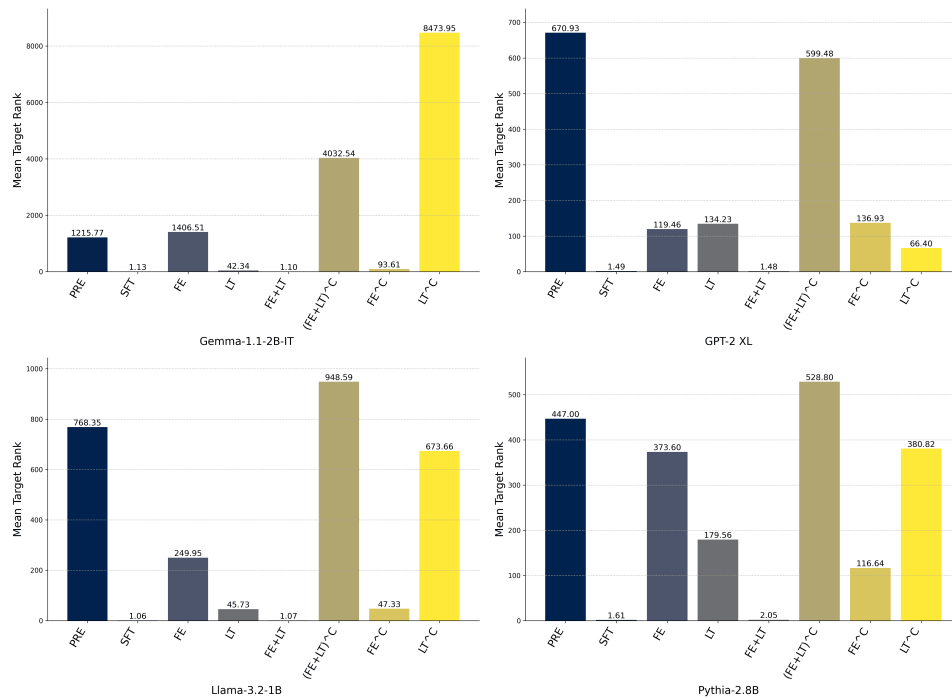


Figure 9: Mean target token rank — Sentence 1

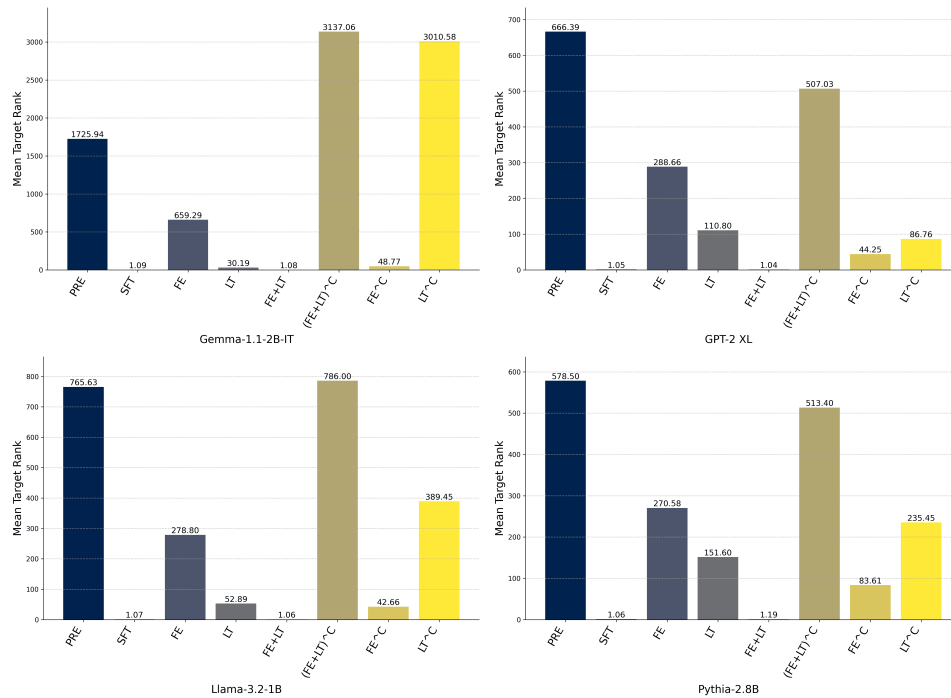


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

We additionally evaluate several values of k on the Fake Movies, Real Actors dataset. The qualitative pattern is invariant to k , though the individual enrichment and recall pathways are weaker at smaller k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

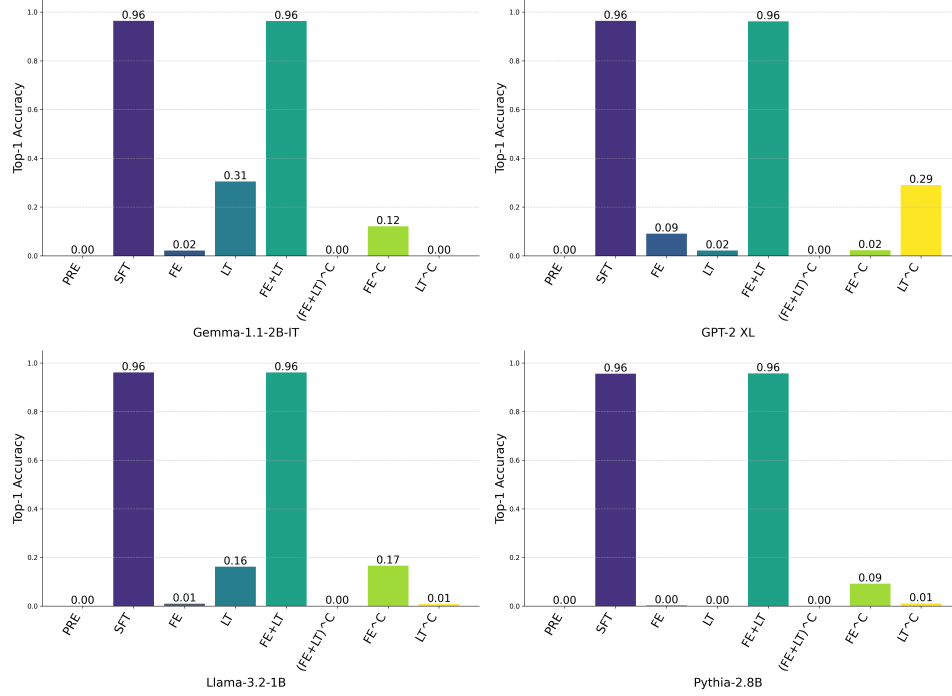


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

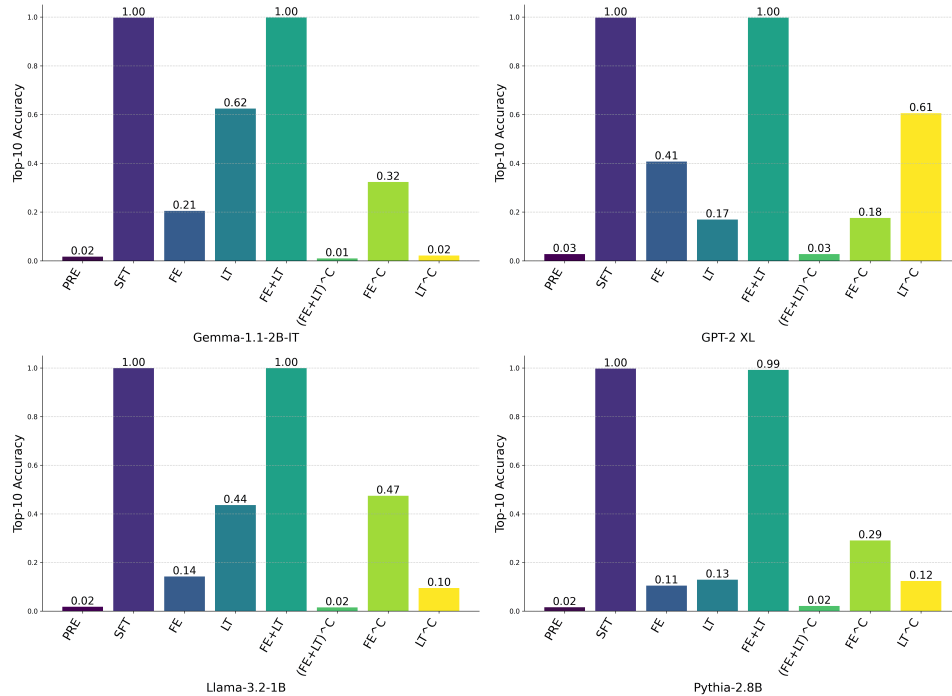


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

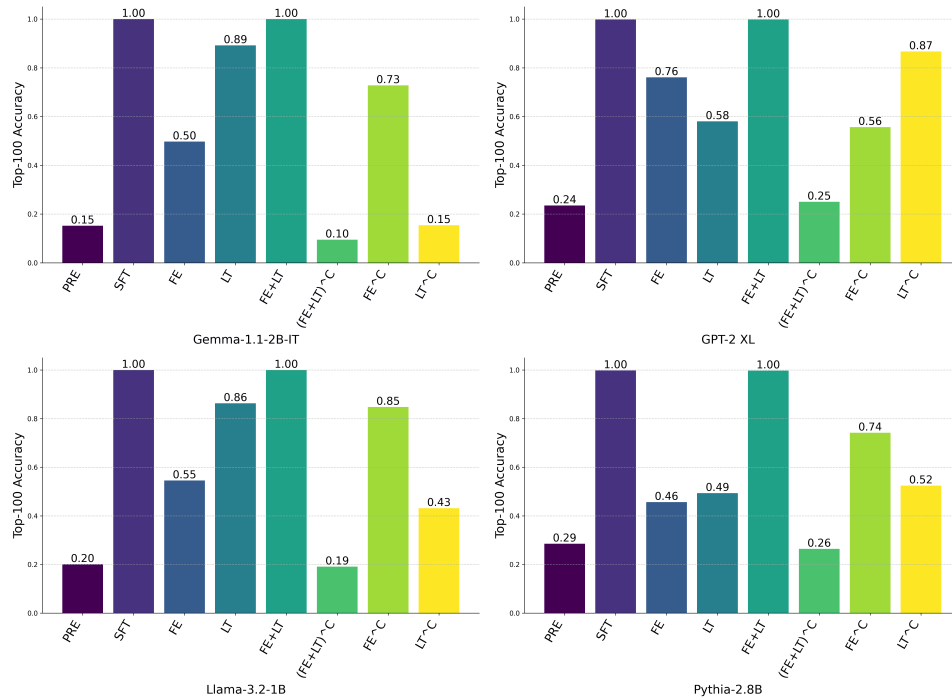


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

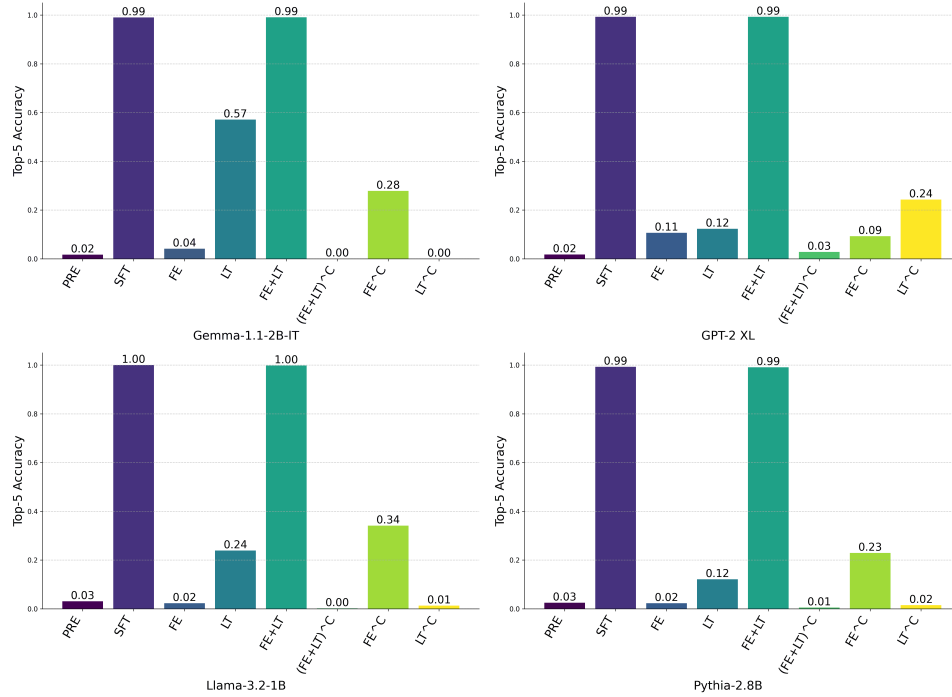


Figure 14: Top-5 accuracy — Sentence 1

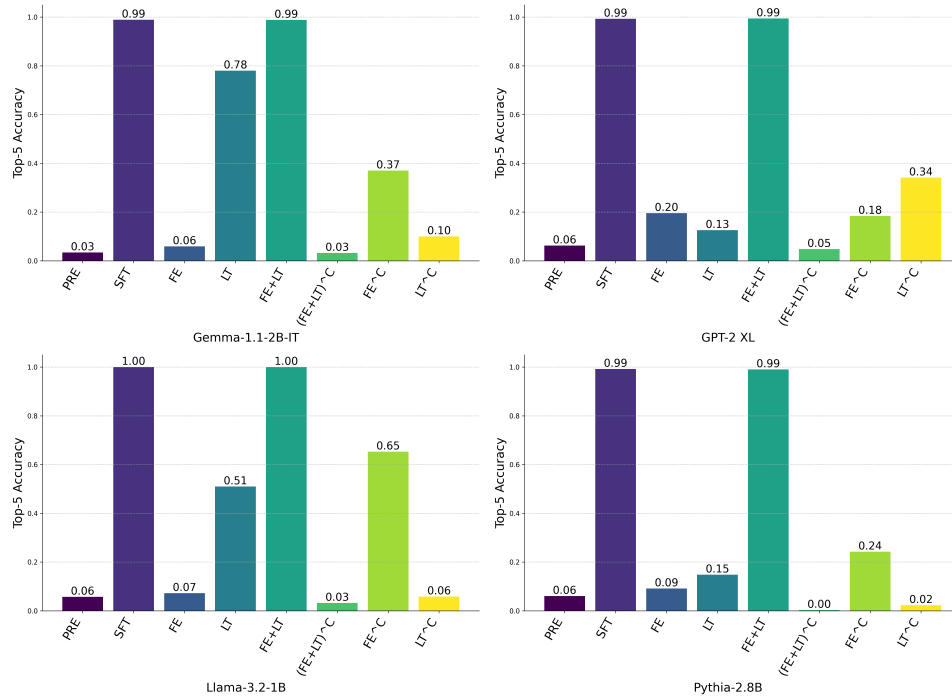


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

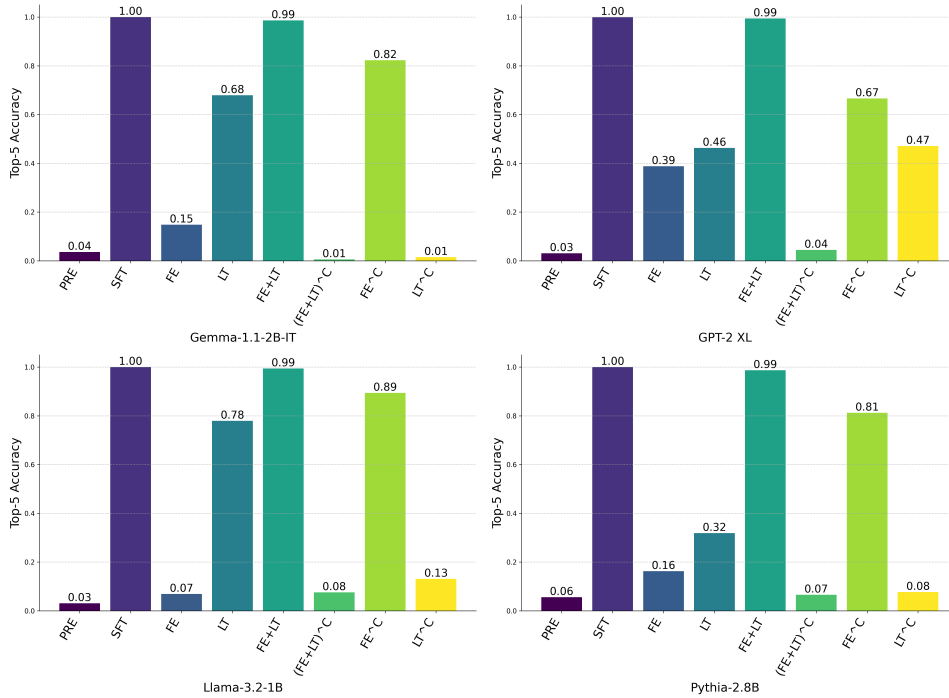


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These results correspond to dynamic weight grafting with the movie title included in the test sentence. The movie title alone is insufficient to recover the correct entity, but combining the movie title with the last token improves accuracy for all models. The combination of the movie title and the first entity is inconsistent across models: relative to the first entity alone, adding the movie title improves GPT-2 XL, leaves Llama 3 and Pythia essentially unchanged, and degrades Gemma. The test sentence is {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}.

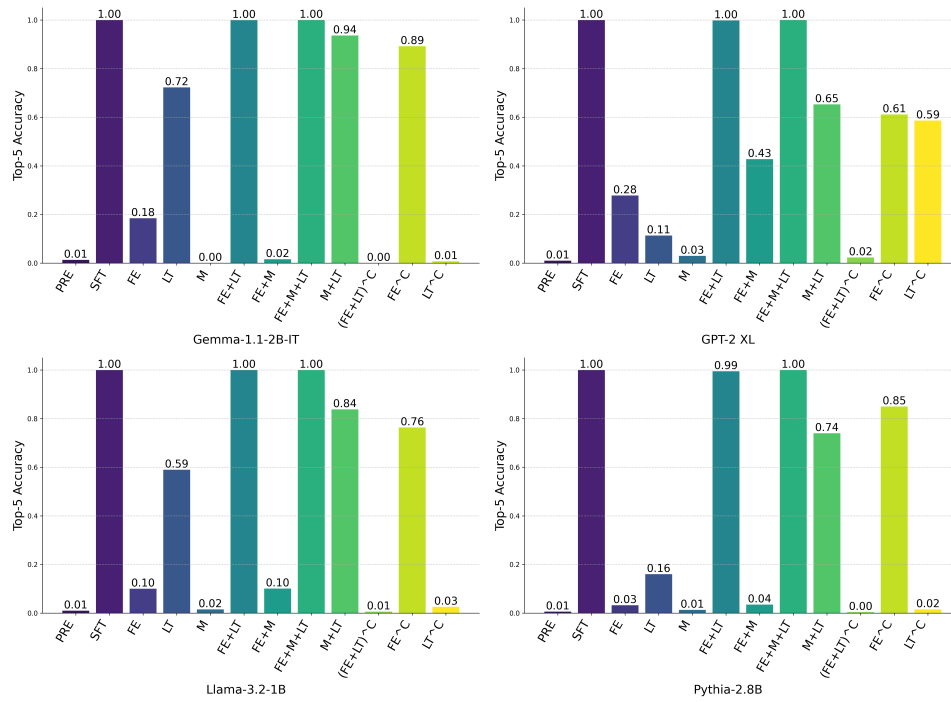


Figure 17: Top-5 accuracy–Sentence 3. “M” refers to the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top- k accuracy shifts somewhat—most noticeably for single-token-position configurations—the overall pattern matches that obtained with the pretrained unembeddings.

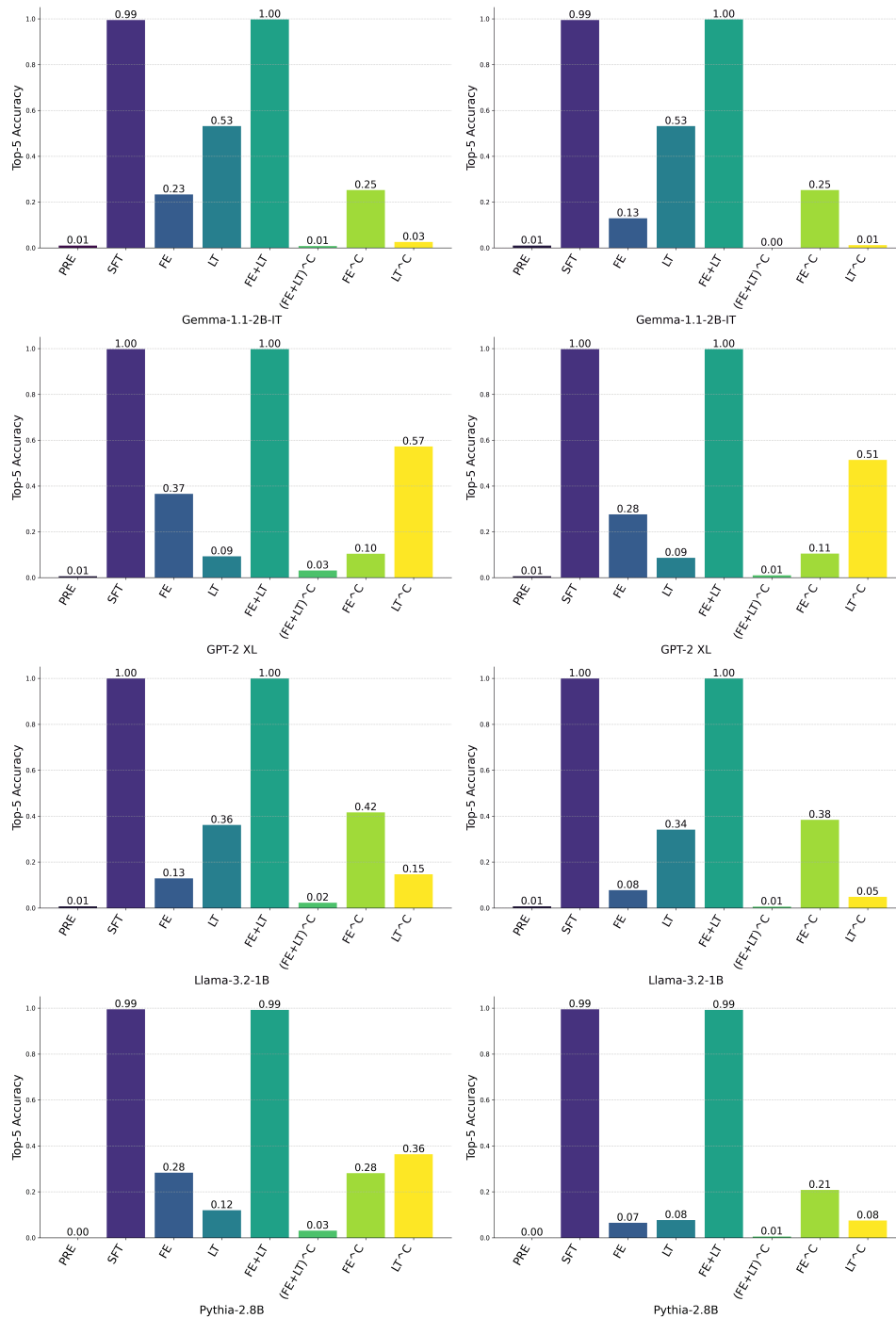


Figure 18: Top-5 accuracy, Sentence 1. Finetuned unembeddings (left) versus pretrained unembeddings (right). The two settings agree closely, except that the finetuned unembeddings yield higher top-5 accuracy in the first-entity-only configuration.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments, which use a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB. The pattern matches our other results, with weaker individual extraction and recall pathways.

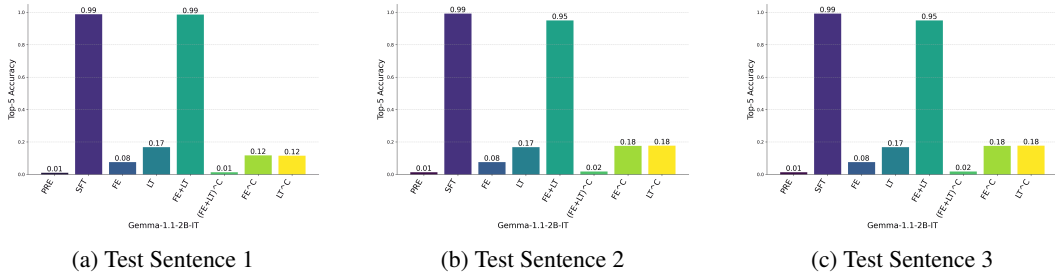


Figure 19: Top-5 accuracy for Gemma finetuned with a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline results for component-grafting experiments in the SFT-to-pretrained direction.

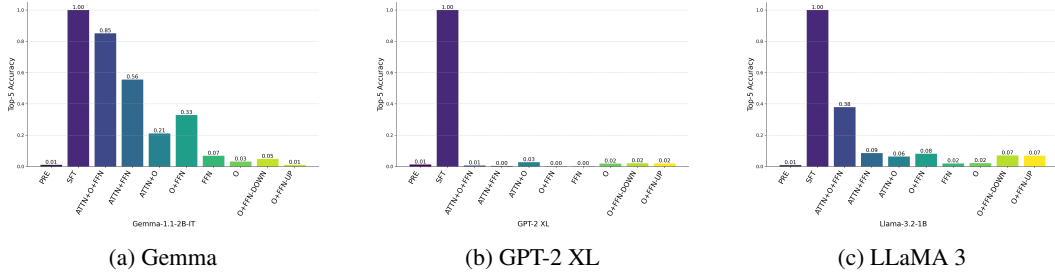


Figure 20: Top-5 Accuracy Component-Grafting Baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

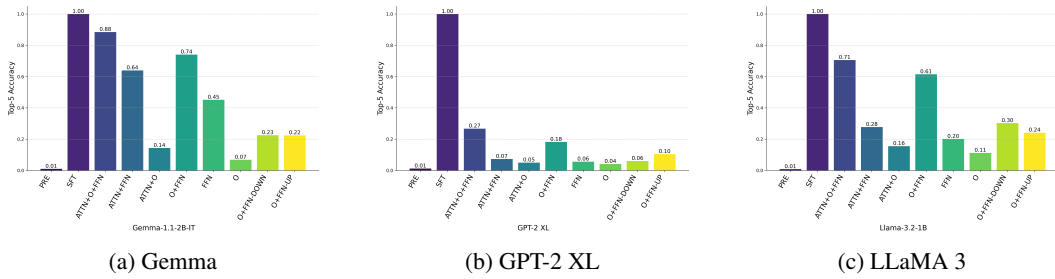


Figure 21: Top-5 Accuracy Reversal Curse Component-Grafting Results — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We report five randomly sampled token-probability results, listing the ten highest-probability tokens
1569 for representative examples. Although our evaluation uses top-5 accuracy, we display the top ten
1570 tokens per example for additional context.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1653

1654 Example 2:

1655 Target: Uta: 0.008

1656 John: 0.901

1657 Joan: 0.022

1658 Sally: 0.011

1659 Chelsea: 0.009

1660 Uta: 0.008

1661 Tyne: 0.003

1662 Gina: 0.002

1663 Elle: 0.001

1664 Lisa: 0.001

1665 Rosemary: 0.001

1666

1667 -----

1668

1669 Example 3:

1670 Target: Jennifer: 0.706

1671 Jennifer: 0.706

1672 John: 0.094

1673 Il: 0.028

1674 Me: 0.020

1675 Stephen: 0.015

1676 Carol: 0.012

1677 S: 0.008

1678 David: 0.008

1679 Elle: 0.007

1680 T: 0.006

1681

```

1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026

```

```

1688 -----
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----

```

C.10.3 GPT-2 XL: (FE+LT)^C

```

1703 Example 1:
1704 Target: Fre: 0.000
1705 her: 0.037
1706 John: 0.018
1707 Jason: 0.017
1708 Chris: 0.016
1709 Adam: 0.015
1710 Zach: 0.014
1711 Josh: 0.013
1712 the: 0.013
1713 Michael: 0.013
1714 Ben: 0.012

```

```

1715 -----
1716 Example 2:
1717 Target: A: 0.002
1718 her: 0.043
1719 the: 0.017
1720 John: 0.016
1721 Tom: 0.016
1722 Peter: 0.012
1723 fellow: 0.012
1724 Michael: 0.011
1725 David: 0.011
1726 James: 0.011
1727 Jack: 0.010

```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1740 -----
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1753 -----
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
1766 -----
```

1767 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 The following is a characteristic result (Gemma grafted at the first entity token) in which the model
1769 concentrates high probability on common tokens while nonetheless placing the target token within
1770 the top 5.

```
1771 -----
1772 Target: Robert: 0.016
1773 a: 0.365
1774 an: 0.151
1775 the: 0.091
1776 Robert: 0.016
1777 two: 0.009
1778 his: 0.008
1779 other: 0.006
1780 John: 0.005
1781 Morgan: 0.005
1782 my: 0.005
```